Real-Time Depth Estimation

Cas Truyers, Niels Verhoeve

Mei 2026

Samenvatting

Dit rapport beschrijft de ontwikkeling van een real-time depth estimation applicatie in CUDA op de ZED BOX (Jetson Orin Nano) met de ZED X stereocamera. We bespreken het theoretisch werkingsprincipe, het traject, optimalisaties en de resultaten.

1 Hardware & Setup

1.1 Hardware Configuratie

We maken gebruik van de ZED BOX en de ZED X stereo camera. Meer informatie over de hardware kan je hier vinden: https://www.stereolabs.com/docs/embedded/zed-box-orin/installation.

1.2 Setup

Voor de codering maken we gebruik van CUDA met cpp. De opstart verliep niet zonder problemen. De ZED X camera werd initieel niet gedetecteerd door het besturingssysteem op de Jetson en de geïnstalleerde ZED applicaties. We vermoeden dat dit een driver/dependencies probleem was aangezien `sudo apt update`, `upgrade` de camera werkend heeft gekregen. Applicaties zoals ZED_Diagnostic en ZED_Explorer gaven ons een manier om de werking te verifiëren.

Daarnaast is het belangrijk om de GPU-klok vast te zetten op een bepaalde frequentie, in dit geval 816 MHz. Hierdoor zijn de latency metingen niet afhankelijk van de frequency scaling door de GPU. Dit doen we met volgende commando's.

```
echo 816000000 | sudo tee /sys/class/devfreq/17000000.ga10b/min_freq
echo 816000000 | sudo tee /sys/class/devfreq/17000000.ga10b/max_freq
sudo jetson_clocks --show
```

2 Goals en scope

Het doel van ons project is om real-time depth-estimation uit te voeren op stereo beelden. In tegenstelling tot een CPU met een superscalar architectuur, die meerdere verschillende instructies binnen één thread parallel uitvoert, maakt een GPU gebruik van een SIMT (Single Instruction, Multiple Threads) architectuur. Hierbij voeren meerdere threads gelijktijdig dezelfde instructie uit op verschillende data. Omdat bij depth-estimation op elke pixel dezelfde operatie wordt toegepast, leent dit probleem zich uitstekend voor deze vorm van parallelisme. Hierdoor kunnen we aanzienlijke tijdswinst behalen ten opzichte van een CPU-implementatie. We onderzoeken de effecten van verschillende parameters op de latency om een optimale balans tussen FPS (∼1/latency) en kwaliteit van de diepte-informatie te bekomen.

3 CPU implementatie

Als startpunt schreven we een CPU-implementatie om later het geoptimaliseerde programma mee af te toetsen. Dit maakt ook voor een goede eerste kennismaking met het project.

Aangezien dit niet de main focus is van het project hebben we gekozen een balans te zoeken tussen snelheid en weergegeven informatie, tot een bruikbaar programma. Dit omvat de edge detection en de depth sensing. De CPU implementatie werkt volgens verwachting aan ongeveer 3 Fps, met een accurate diepteschatting. Het aantal FPS is verder afhankelijk van factoren zoals CPU-gebruik, image complexity, en andere factoren. Dit toont aan dat er zeker een optimalisatie



Figuur 1: Edge en depth sensing applicatie voor CPU

aan de hand van de GPU mogelijk is, zoals voorspeld in de introductie. Onze belangrijkste metric is latency ($\sim 1/\text{FPS}$), voor deze toepassing is deze gemiddeld 313,925 ms ($\approx 3,192$ FPS).

Voor de code word gebruik gemaakt van de OpenCV en Zed libraries. Hierbij zijn twee verschillende algoritmes geïmplementeerd, adaptive EMA en Temperal Median mode. Daarnaast worden nog enkele filters toegevoegd voor stabiliteit van het beeld, namelijk Bilateral Filtering, SGBM en Texture Variance Analysis. Dit zijn algoritmes complexer dan deze gebruikt voor de GPU code in het vervolg van dit document, maar berustend op dezelfde onderliggende principes. Het gebruik van deze meer complexe algoritmen maakt het mogelijk om een betere depth accuracy en visuele smoothness van het beeld te krijgen. Dit is echter geen goed meetbare metric, en een secundaire metric voor ons doel.

4 GPU implementatie

4.1 Edge Detection als Proof-of-Concept

Als eerste stap implementeerden we een Sobel-filter voor edge detection, toegepast in real-time op de video stream van één van de twee camera's op de ZED X. Dit diende vooral als proof-of-concept om te verifiëren of we succesvol frames konden ophalen via de ZED SDK, deze konden bewerken in een custom CUDA-kernel op de GPU, en het resultaat via OpenCV op het scherm in real-time konden tonen.

4.2 Stereo Disparity Algoritme

Na de edge detection zijn we overgegaan op depth sensing. De kern van het algoritme is het vinden van overeenkomstige pixels in de linker en rechter camerabeelden. Objecten die dichtbij de camera staan, verschuiven sterk horizontaal tussen de twee beelden, terwijl objecten ver weg nauwelijks verschuiven. Deze horizontale verschuiving noemen we de *dispariteit* (d). Om deze dispariteit te berekenen, gebruiken we het *Sum of Absolute Differences* (SAD) algoritme. De werking is als volgt:

1. Neem een matrix van pixels (bijvoorbeeld een 9×9 *window*) rondom een referentiepixel in het linkerbeeld.
2. Verschuif deze matrix iteratief over dezelfde horizontale rij in het rechter beeld.
3. Bereken per stap het verschil tussen de linker- en rechter pixels. De positie met de kleinste totale afwijking (de laagste SAD-waarde) is de meest waarschijnlijke match.

Dit gehele proces moet voor elke pixel opnieuw worden uitgevoerd. Zodra de beste dispariteit (d) gevonden is, kan de werkelijke afstand in meter (Z) worden berekend met de formule:

$$Z = \frac{f \cdot B}{d}$$

Waarbij f de brandpuntsafstand van de camera is (in pixels) en B de baseline (de fysieke afstand tussen de linker en rechter camera in meter).

```
for (int d = MIN_D; d < MAX_D; d += D_STEP) {
    float current_sad = 0;
    // Loop over de window matrix
    for (int wy = -WIN; wy <= WIN; wy++) {
        for (int wx = -WIN; wx <= WIN; wx++) {
            float l = (float)rowL[x + wx];
            float r = (float)rowR[x + wx - d];
            current_sad += fabsf(l - r);
        }
    }
    // Bewaar de kleinste error
    if (current_sad < min_sad) {
        min_sad = current_sad;
        best_disp = d;
    }
}
```

Listing 1: De kern van de SAD berekening (in Global Memory)

4.3 Camera Resolutie

In een eerste niet-geoptimaliseerde versie zagen we een extreme latency en een throughput van slechts ∼1 FPS. Een voor de hand liggende oplossing is het downscalen van de input-resolutie voordat we deze aan de kernel geven. Hoewel dit effectief was, hebben we besloten om terug naar de native 1080p resolutie te gaan en andere technieken te gebruiken om de performantie te verbeteren. Het doel was om de latency te minimaliseren door *optimalisaties*, zonder de Diepte-informatie al te veel op te offeren. (Een lagere resolutie komt overeen met minder punten in de afbeelding om nauwkeurig te kunnen matchen.)

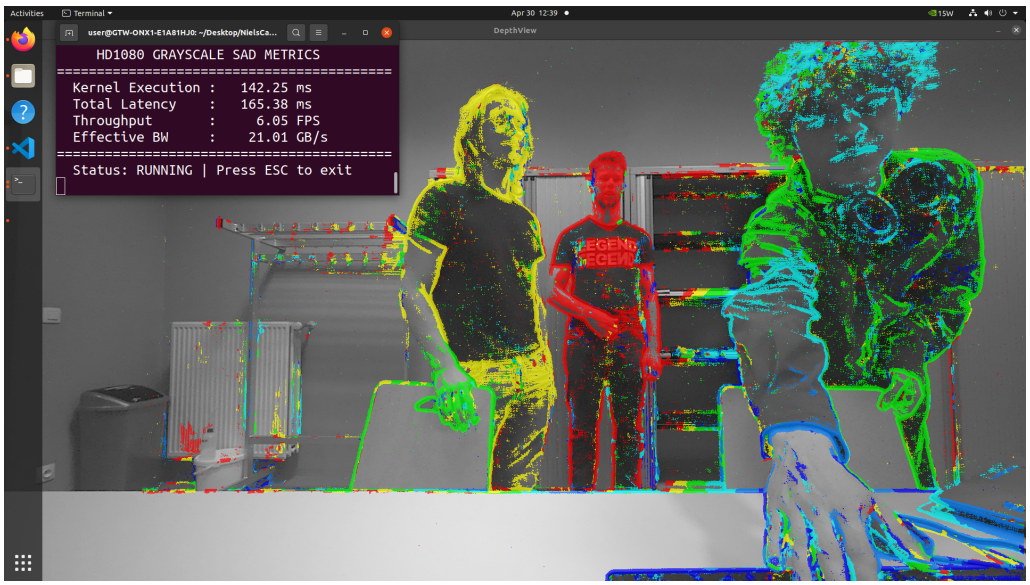
4.4 Color-Depth Mapping: Look-Up-Table in Constant Memory

Voor het visueel weergeven van de diepte-informatie maken we gebruik van een kleurschaal en een visuele overlay (zoals in figuur 1 en 3). Initieel werden de kleuren gemapped aan de diepte met een if-else statement binnen de kernel. We definieerden bepaalde ranges aan 1 enkele kleur (blauw, groen, geel, rood), alle pixels die binnen een range vielen kregen dan dezelfde kleur. Hierdoor verloren we veel informatie in het beeld.

Om dit op te lossen, zijn we overgeschakeld op een Look-Up Table (LUT). Op de CPU genereren we een array van 256 elementen, waarbij elke index (0-255) overeenkomt met een RGB gemapte kleurwaarde. Deze LUT kopiëren we dan éénmalig naar het snelle constant memory van de GPU. Tijdens de executie is de mapping van dispariteit naar kleur nu gereduceerd tot een simpele indexering: `color = c_lut[best_disp];` wat een betere diepteresolutie geeft en overbodige if-else statements uit de kernel haalt.

4.5 Van RGB naar Grayscale

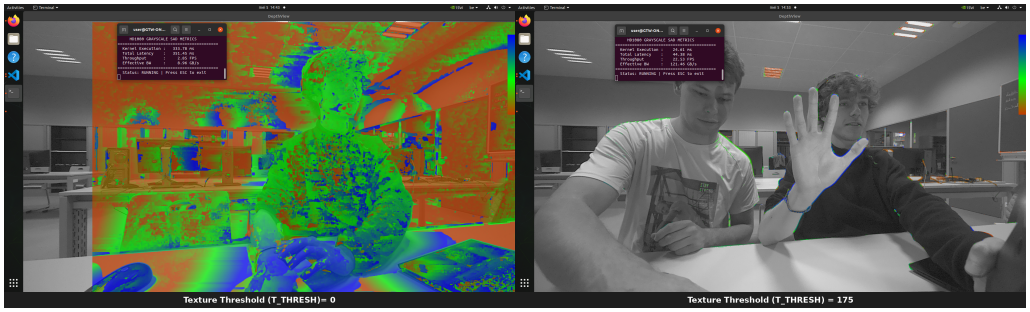
De meest significante initiële prestatiewinst kwam door de input van de camera om te zetten van RGB naar Grayscale. In een RGBA-formaat bestaat elke pixel uit 4 bytes. Voor een 9×9 matrix-vergelijking veroorzaakt dit een enorme bottleneck op de geheugenbus. Door over te stappen op een 1-byte grayscale representatie, reduceerden we de vereiste geheugen bandbreedte met ∼75%. Deze aanpassing bracht de performance direct van ∼1 naar 4 FPS, zonder verlies van diepte-informatie.



Figuur 2: Resultaat met grayscale en texture threshold

4.6 Texture Threshold

Het SAD stereo matching algoritme faalt op volledig egale vlakken (zoals een lege witte muur), omdat de matrix overal exact dezelfde foutmarge oplevert. Om dit op te lossen, en tegelijk rekenkracht te besparen, introduceerden we een *texture threshold* (`T_THRESH`). Voordat de zware SAD-loop start, berekenen we het contrast van het venster. Als het oppervlak te vlak is, slaan we de SAD-berekening volledig over en behouden we de gray-scale waarden van de pixel in de output. Dit gaf zowel een voordeel op vlak van latency als op de interpreteerbaarheid van het beeld.

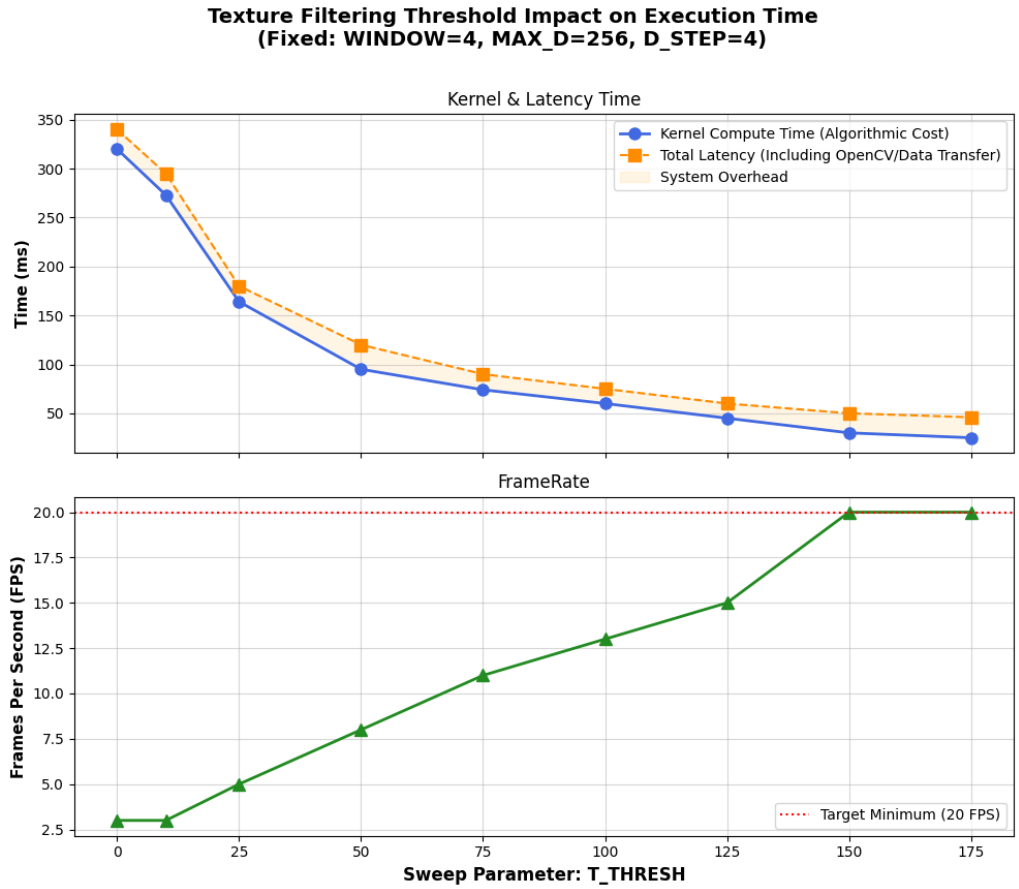


Figuur 3: Vergelijking van diepte-informatie bij T_THRESH 0 en T_THRESH 175

- **Visuele impact:** Zoals te zien op Figuur 3 wordt bij een T_THRESH van 0 de diepte voor elke pixel berekend, we krijgen dus een volledig ingekleurde overlay. Uiterst links zien we hier geen overlay omdat de rechter camera dit deel niet ziet, en de overlay word toegepast op het linker camerabeeld.

  Als we de threshold laten toenemen tot 175 dan zien we dat het een soort strenge edge detection uitvoert (door de sterke overgang van contrast). Ook zien we dat bepaalde objecten met patroon-gewijs contrast (bv. lamp aan plafond) die net binnen de vergelijkende texture window valt ook als hoog-textuur wordt beschouwd. Algemeen is een minimum aan texture threshold aangeraden om de ruis op egale oppervlakken weg te halen, wat het visueel makkelijker maakt om het beeld af te lezen.

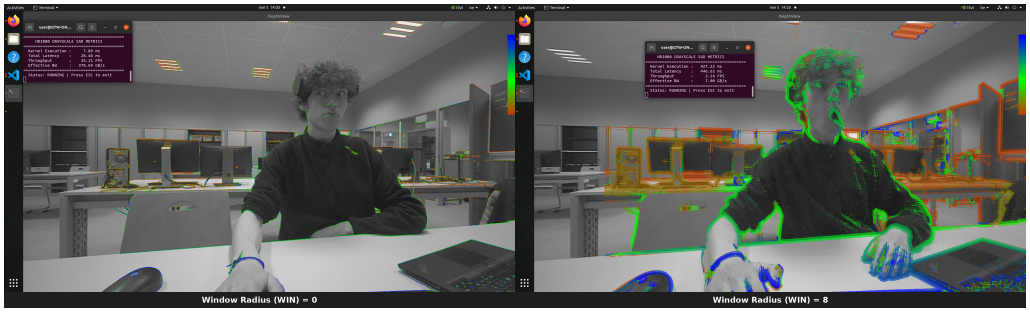
- **Performance impact:** Zoals te zien op Figuur 4 heeft een kleine threshold al aanzienlijk effect op de latency. Dit komt omdat de diepte voor de grootste egale oppervlakken, zoals muren, niet meer berekend worden. Afhankelijk van het aantal egale oppervlakken kan dit een aanzienlijk effect hebben op de latency. Ook voor performantie is een minimum aan texture threshold dus aangeraden. Naarmate de texture threshold groter wordt zien we steeds minder effect op de latency, dit is volgens verwachting aangezien de meeste effen pixels initieel al worden weggefilterd.



Figuur 4: Performantie bij verschillende T_THRESH

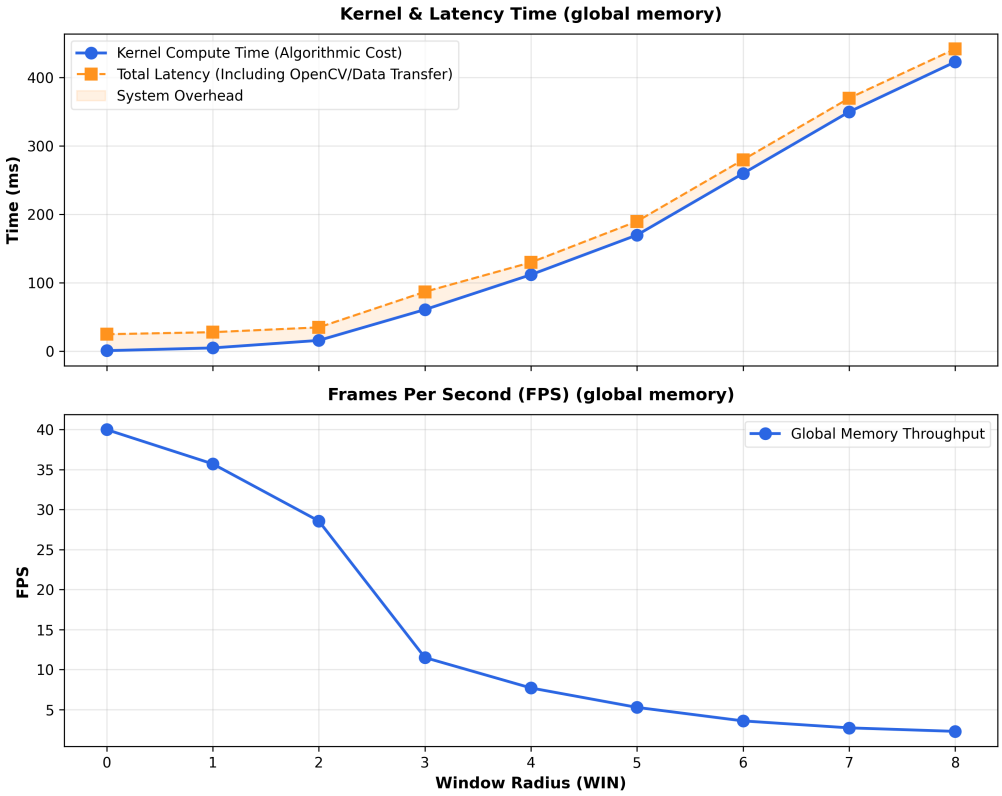
4.7 Window Radius (WIN)

De `WIN`-parameter (window radius) bepaalt de grootte van de matrix die wordt vergeleken in het SAD algoritme (bv. een radius van 4 betekent een 9×9 matrix). Een grotere window bevat bijgevolg meer context en textuurinformatie.



Figuur 5: Vergelijking van diepte-informatie bij WIN 0 en WIN 8

- **Visuele impact:** Een klein window (zoals 3×3) is gevoeliger voor noise en foutieve matches. Een grotere window (9×9) produceert een stabiele en zuivere dieptemap, maar verzacht scherpe objectranden in de diepte. Met andere woorden, de edges die gekleurd worden zullen dikker zijn.
- **Performance impact:** De rekentijd neemt toe met de window size, omdat meer berekeningen nodig zijn.



Figuur 6: Performantie bij verschillend Window Sizes voor global memory

4.8 Shared vs Global Memory

Threads die pixels op dezelfde horizontale positie verwerken, moeten dezelfde x-as doorlopen tijdens de stereo matching. Het herhaaldelijk inlezen van deze identieke data uit het tragere global memory vormt een potentiële bottleneck voor de databus. Om dit op te lossen hebben we shared memory tiling toegepast. Elke block van 16×16 threads laadt een grote tegel in het on-chip shared geheugen. Deze tegel bevat zowel de interne pixels als de omliggende "halo"pixels die nodig zijn voor de matrixradius (WIN) en de maximale verschuiving (MAX_D).

```
// Allocatie van de tile in Shared Memory
__shared__ uint8_t s_left[S_LEFT_H * S_LEFT_W];

// Elke thread in het block laadt een deel van de tegel in
for (int i = tid; i < S_LEFT_H * S_LEFT_W; i += num_threads) {
    int r = i / S_LEFT_W;
    int c = i % S_LEFT_W;
    int gy = halo_y + r;
    int gx = halo_left_x + c;

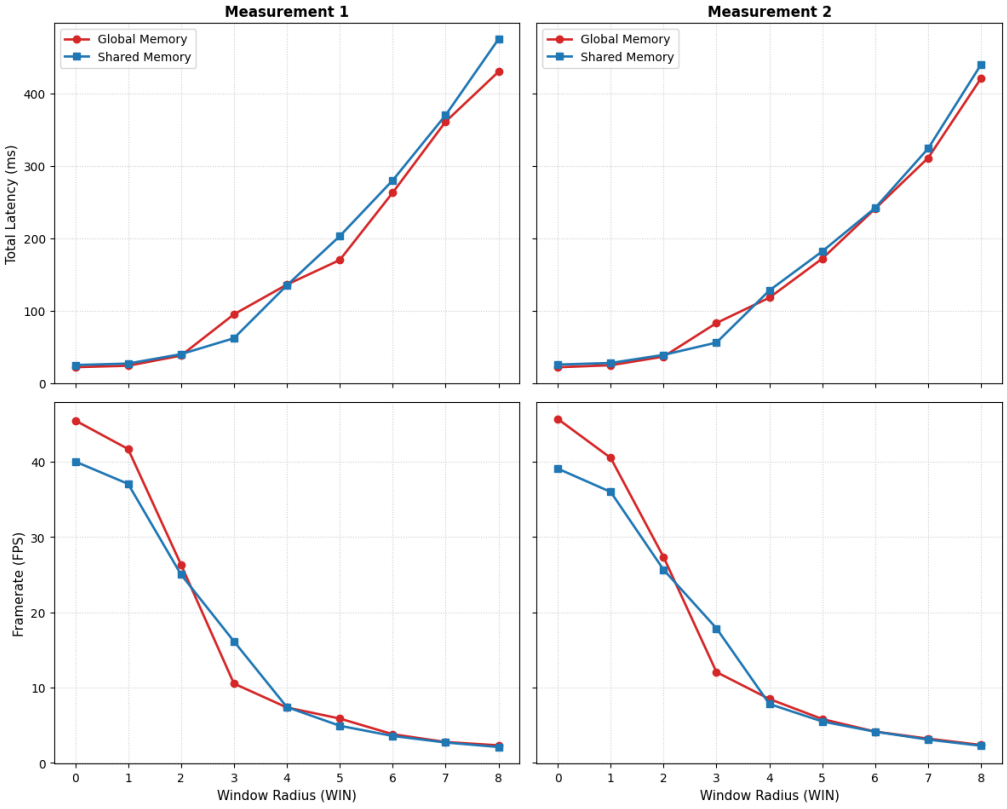
    // Bounds check voor de randen van het beeld
    if (gx >= 0 && gx < w && gy >= 0 && gy < h)
        s_left[i] = left[gy * step_gray + gx];
    else
        s_left[i] = 0;
}
// wachten tot de volledige tile is ingeladen
__syncthreads();
```

Listing 2: inladen van data naar Shared Memory

Om de mogelijke prestatiewinst van shared memory ten opzichte van global memory te demonstreren, hebben we opnieuw de window-radius (WIN) parameter doorlopen omdat dit direct de mate van data-overdracht en overlap tussen de naburige threads bepaald. De eerste meting toonde geen groot en consistent verschil tussen shared en global memory, daarom is er een tweede meting met extra zorgvuldigheid uitgevoerd. Deze resultaten zijn te zien in figuur 7. We stellen dezelfde trend vast in de tweede meting en concluderen het volgende.

- **Geen winnaar door overhead:** Voor slechts 1 size is de shared memory duidelijk sneller, wel zien we dezelfde trend in beide metingen wat een meetfout uitsluit. Aangezien we bij onze tiling gebruik maken van een blokkende syncthreads() kan dit eventueel een overhead geven waardoor shared memory in slechts bepaalde gevallen (window sizes) voordelig is.
- **Shared memory is optimaal bij WIN=3:** Onze resultaten tonen een consistente winst voor shared memory bij een radius van 3 (7x7 pixel window). Dit is waarschijnlijk de winst van de shared memory tegenover de overhead.
- **Experimenteren met block-sizes:** Naast de standaard 16x16 configuratie is er geëxperimenteerd met andere block-vormen, zoals 32x8. De verwachting was dat een breder block het horizontale data-hergebruik in shared memory zou verhogen, aangezien de SAD-loop enkel de x-as doorloopt leek dit logisch. In de praktijk resulteerde dit echter in een hogere latency. We vermoeden dat de architectuur efficiënter om kan gaan met 16x16 blokken.

Algemeen concluderen we dat de globale kernel verrassend sterk presteert. Dit kan een resultaat zijn van de moderne cache-hiërarchie van de jetson nano, en automatische kernel optimalisaties. Echter presteert shared memory wel beter bij een 7x7 window, wat algemeen ook een goede window size is.



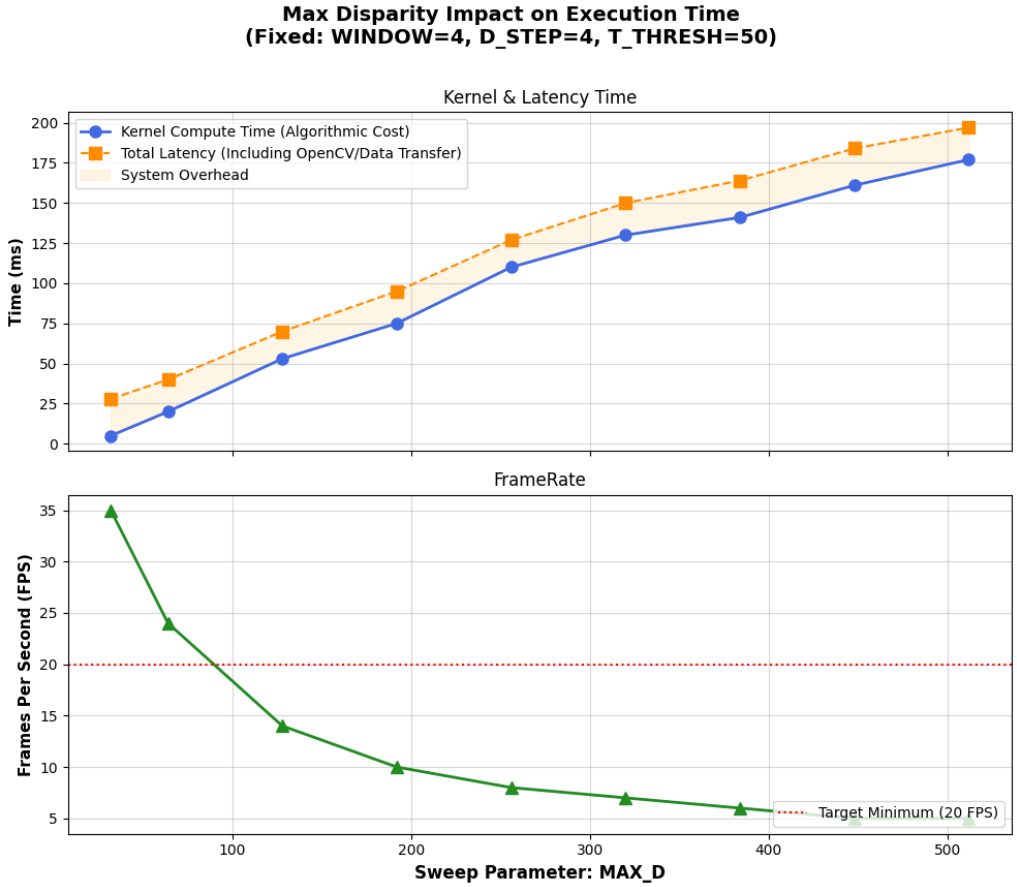
Figuur 7: Extra metingen om Shared vs Global te vergelijken

4.9 Max Disparity (MAX_D)

De `MAX_D` parameter definieert de maximale horizontale pixelverschuiving die het algoritme uitvoert. Dit bepaalt de minimale meetafstand aangezien een punt dichter bij de camera een grotere dispariteit (verschuiving) heeft.

- **Werking:** Als `MAX_D` op 256 staat, zoekt de matrix maximaal 256 pixels naar rechts. Omdat objecten dichterbij de camera sneller horizontaal verschuiven over de twee beelden, staat een hogere `MAX_D` gelijk aan het "dichterbij kunnen kijken".
- **Visuele impact:** Bij een lage `MAX_D` (bv. 64) verdwijnen objecten die zich op minder dan de threshold, omgerekend in meter, van de camera bevinden; ze krijgen geen dieptekleur meer omdat hun dispariteit buiten de toegestane zoekmarge valt.

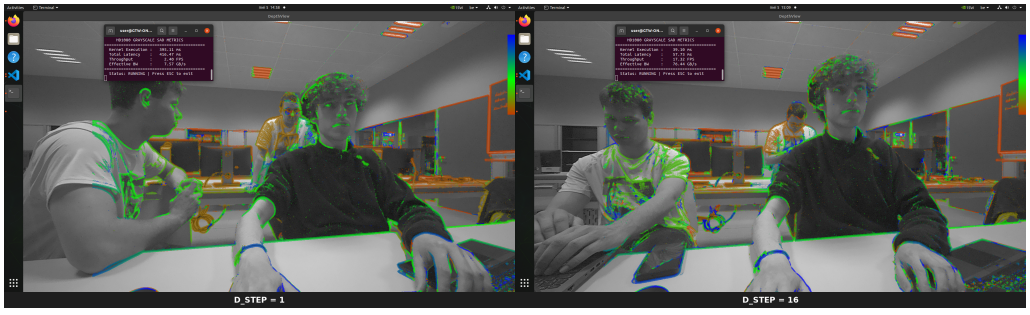
- **Performance impact:** De rekentijd schaalt proportioneel (lineair) met de grootte van de dispariteit. Een halvering van `MAX_D` (bijvoorbeeld van 256 naar 128) resulteert ruwweg in een halvering van de kernel rekentijd.



Figuur 8: Performantie bij verschillend Maximum Dispariteit

4.10 Disparity Step (D_STEP)

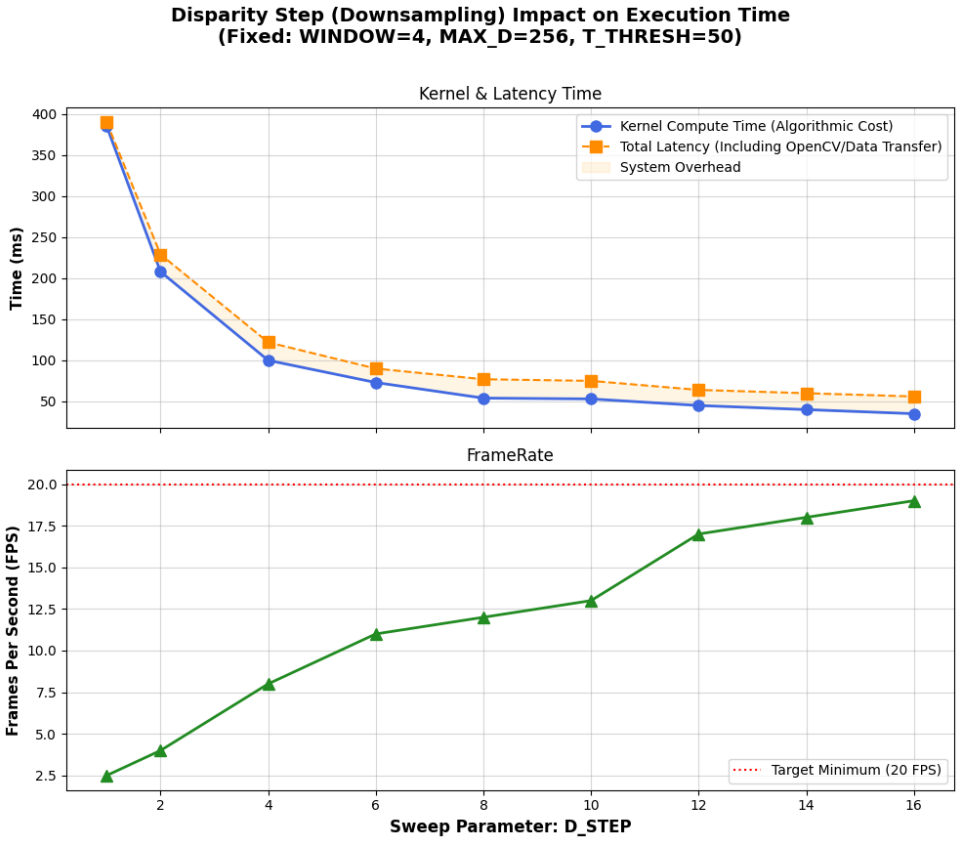
Door een stapgrootte (disparity step of stride) groter dan 1 te gebruiken in onze SAD loop, passen we een vorm van sub-sampling toe in de diepte. Dit verhoogt de FPS ten koste van de diepte-resolutie.



Figuur 9: Vergelijking van diepte-informatie bij D_STEP 1 en D_STEP 16

- **Werking:** In plaats van elke mogelijke horizontale verschuiving te controleren ($d = 30, 31, 32 \ldots$), dwingen we het algoritme om stappen over te slaan ($d = 30, 34, 38 \ldots$ bij `D_STEP` $= 4$).
- **Visuele impact:** Een grotere stapgrootte (bv. 8) "kwantiseert"de Diepte-informatie. Kleurovergangen op objecten zullen dus minder continu zijn, maar dit is voor een kleine stapgrootte amper zichtbaar.

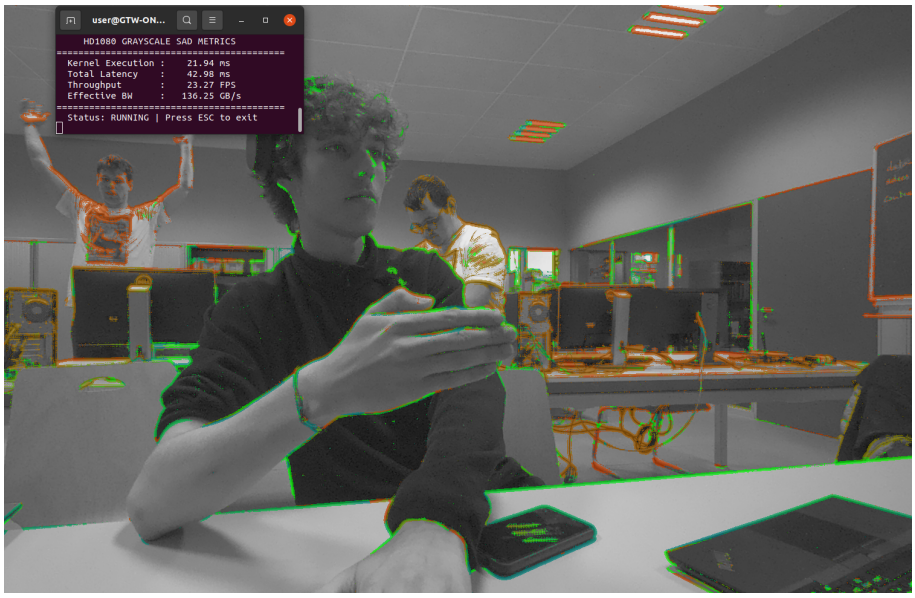
- **Performance impact:** Dit is een zeer efficiënte optimalisatie. Het verhogen van `D_STEP` van 1 (bijna 400 ms rekentijd) naar 4 (ongeveer 100 ms rekentijd) leverde een snelheidsverbetering van een factor 4 op, met een kleine impact op de functionele bruikbaarheid.



Figuur 10: Performantie bij verschillende Disparity Steps

4.11 Resultaat met gebalanceerde parameters

Aan de hand van de verworven inzichten kunnen we parameters kiezen die een goede balans van latency en diepte informatie geven. Figuur 11 toont het resultaat met ∼24 FPS, wat als vloeiend wordt beschouwd door het menselijk oog. Helaas zijn de gebruikte parameters verloren gegaan.



Figuur 11: Resultaat bij gebalanceerde parameters

5 Reflectie

Hoewel de implementatie succesvol real-time depth estimation presenteert, legde het ook de limieten van pixel matching-gebaseerde algoritmes bloot. Zo veroorzaken lampen en repetitieve structuren foutieve matches, en dus foute diepte berekeningen. Ook op andere objecten ondervonden we foutieve matches, wat het resultaat kan zijn van de hoek-afhankelijke reflecties gecreëerd door een object, wat voor de linker en rechter camera kan verschillen.

Om de robuustheid te verbeteren zouden andere cost-metrics (in plaats van SAD) getest kunnen worden, zoals *Zero-mean Normalized Cross-Correlation*. Een relevante volgende stap zou het vergelijken met AI-gebaseerde depth estimation modellen (zoals RAFT-Stereo) zijn.

6 Conclusie

In dit project realiseerden we een real-time depth sensing applicatie die succesvol op een resolutie van 1080p met acceptabele FPS werkt. De initiële 1 FPS hebben we dankzij de besproken optimalisaties naar ∼24 FPS gekregen, terwijl we een goede schatting en visualisatie van de diepte behouden. Door de parallelle SIMT-architectuur van de GPU te benutten, is een grote speed-up gerealiseerd ten opzichte van de CPU implementatie (∼ ×8 sneller).

De belangrijkste conclusies uit onze tests zijn:

- **Optimalisatie:** De combinatie van grayscale-conversie, texture thresholding en sub-sampling (`D_STEP`=4) verhoogde de frame rate aanzienlijk.
- **Shared vs Global Memory:** De *Shared Memory* implementatie is optimaal bij een window-radius van 3 (7x7 window). Verrassend is dat *Global Memory* beter presteert voor andere window sizes.
- **Parameters:** Het aanpassen van parameters kan een groot effect hebben op zowel de frame rate als de diepte informatie. De `MAX_D` parameter heeft een lineaire impact op de latency, terwijl de `D_STEP` parameter de meest efficiënte methode bleek om de frame rate te verhogen zonder een drastisch verlies van diepte informatie. Een minimale texture threshold is essentieel om (ruis van) egale oppervlakken uit te sluiten.

7 Bijlagen

De broncode is beschikbaar op: https://github.com/Niels-V-KU/Lab_Geavanceerde_Computerarchitectuur/tree/main/projectZED